

Name-Nancy

Subject-Computer Vision Lab

CUID- CU240251005

✓ Experiment No. 4

Experiment Name :- Frequency Domain Image Filtering using Fourier Transform

1. Import the required libraries and load a grayscale image for processing.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("/image.webp", cv2.IMREAD_GRAYSCALE)
if img is None:
    print("Error: Image not found")
else:
    print("Image loaded successfully")
    print("Image shape:", img.shape)
plt.imshow(img, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")
plt.show()
```

Image loaded successfully
Image shape: (599, 900)

Original Grayscale Image



2. Compute the Discrete Fourier Transform (DFT) of the image using OpenCV or NumPy.

```
img_float = np.float32(img)
print("Image converted to float32")
dft = cv2.dft(
    img_float,
    flags=cv2.DFT_COMPLEX_OUTPUT
)
print("DFT calculated successfully")
print("DFT shape:", dft.shape)
```

Image converted to float32
DFT calculated successfully
DFT shape: (599, 900, 2)

3. Shift the zero-frequency component to the center of the frequency spectrum for visualization.

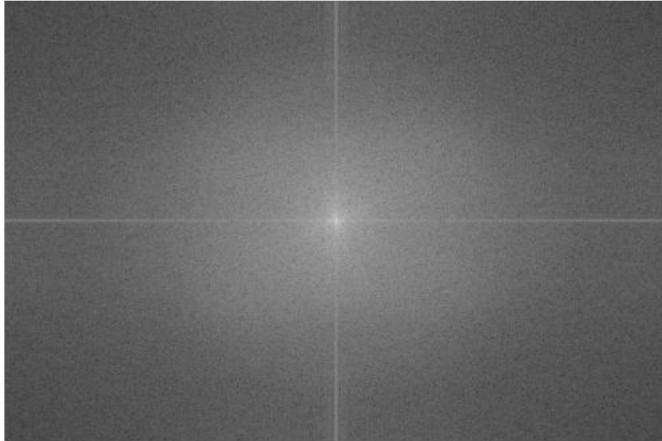
```
dft_shift = np.fft.fftshift(dft)
print("Zero-frequency component shifted to center")
```

Zero-frequency component shifted to center

4. Display and analyze the magnitude spectrum of the transformed image.

```
magnitude = cv2.magnitude(
    dft_shift[:, :, 0],
    dft_shift[:, :, 1]
)
magnitude_spectrum = 20 * np.log(magnitude + 1)
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Magnitude Spectrum")
plt.axis("off")
plt.show()
```

Magnitude Spectrum



5. Design and apply a Low-Pass Frequency Filter to suppress high-frequency components and reduce image noise.

```
rows, cols = img.shape
crow, ccol = rows // 2, cols // 2

radius = 50

y, x = np.ogrid[:rows, :cols]
distance = np.sqrt((x - ccol)**2 + (y - crow)**2)

mask_lpf = np.zeros((rows, cols, 2), np.float32)
mask_lpf[distance <= radius] = 1

filtered_lpf = dft_shift * mask_lpf

print("Low-Pass Filter applied")
plt.subplot(2, 2, 3)
plt.imshow(lpf_image, cmap="gray")
plt.title("Low-Pass Filtered Image")
plt.axis("off")
```

Low-Pass Filter applied
(np.float64(-0.5), np.float64(899.5), np.float64(598.5), np.float64(-0.5))

Low-Pass Filtered Image



6. Design and apply a High-Pass Frequency Filter to suppress low-frequency components and enhance edges and fine details.

```
mask_hpf = np.ones((rows, cols, 2), np.float32)
mask_hpf[distance <= radius] = 0

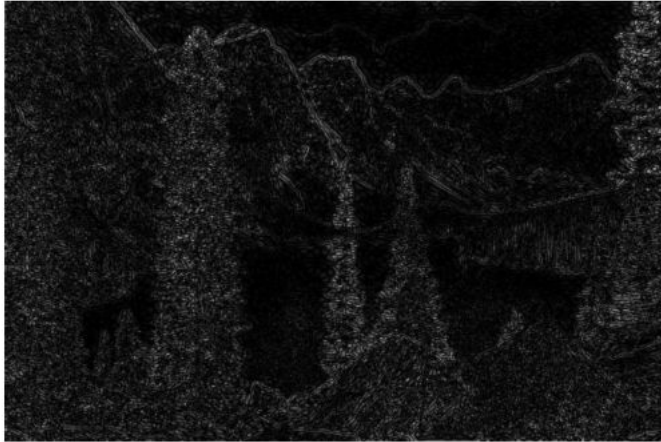
filtered_hpf = dft_shift * mask_hpf

print("High-Pass Filter applied")
plt.subplot(2, 2, 4)
plt.imshow(hpf_image, cmap="gray")
plt.title("High-Pass Filtered Image")
```

```
plt.axis("off")
plt.imshow(hpf_image, cmap="gray")
plt.title("High-Pass Filtered Image")
plt.axis("off")
```

High-Pass Filter applied
 (np.float64(-0.5), np.float64(899.5), np.float64(598.5), np.float64(-0.5))

High-Pass Filtered Image



7. Perform the Inverse Fourier Transform (IDFT) to reconstruct the filtered images in the spatial domain.

```
# Low-Pass IDFT
filtered_lpf = np.fft.ifftshift(filtered_lpf)
lpf_image = cv2.idft(filtered_lpf)
lpf_image = cv2.magnitude(
    lpf_image[:, :, 0],
    lpf_image[:, :, 1]
)

# High-Pass IDFT
filtered_hpf = np.fft.ifftshift(filtered_hpf)
hpf_image = cv2.idft(filtered_hpf)
hpf_image = cv2.magnitude(
    hpf_image[:, :, 0],
    hpf_image[:, :, 1]
)

print("Inverse Fourier Transform completed")
```

Inverse Fourier Transform completed

8. Compare the original image, frequency spectrum, low-pass filtered image, and high pass filtered image.

```
lpf_image = cv2.normalize(
    lpf_image, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)

hpf_image = cv2.normalize(
    hpf_image, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)

plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Magnitude Spectrum")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(lpf_image, cmap="gray")
plt.title("Low-Pass Filtered Image")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(hpf_image, cmap="gray")
```

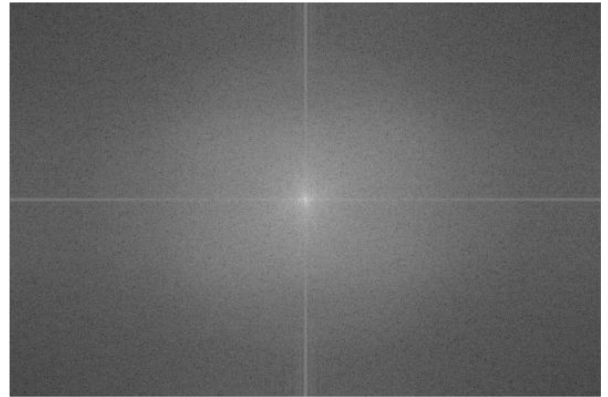
```
plt.title("High-Pass Filtered Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Image



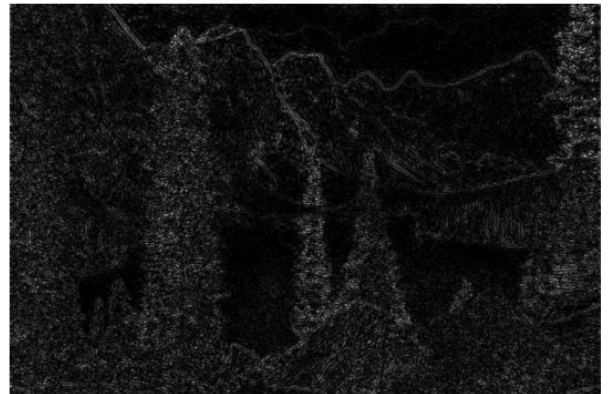
Magnitude Spectrum



Low-Pass Filtered Image



High-Pass Filtered Image



9. Analyze the impact of frequency domain filtering on image smoothness, edge enhancement, and noise removal.

```
print("Analysis:")
print("Low-Pass Filter: Smooths the image and reduces high-frequency details/noise.")
print("High-Pass Filter: Enhances edges and fine details.")
print("Low-Pass result is smoother than the original image.")
print("High-Pass result highlights boundaries and fine textures.")
```

Analysis:
 Low-Pass Filter: Smooths the image and reduces high-frequency details/noise.
 High-Pass Filter: Enhances edges and fine details.
 Low-Pass result is smoother than the original image.
 High-Pass result highlights boundaries and fine textures.

10. Document the observations and discuss the advantages of frequency domain filtering in practical computer vision applications

```
print("Observations:")
print("1. DFT converts the spatial image into frequency components.")
print("2. Low frequencies are located near the center of the shifted spectrum.")
print("3. Low-Pass Filtering suppresses high-frequency components.")
print("4. High-Pass Filtering suppresses low-frequency components.")
print("5. Low-Pass Filtering is useful for smoothing and noise reduction.")
print("6. High-Pass Filtering is useful for edge enhancement and feature extraction.")
print("7. Frequency-domain filtering provides selective control over image frequencies.")
```

Observations:
 1. DFT converts the spatial image into frequency components.
 2. Low frequencies are located near the center of the shifted spectrum.
 3. Low-Pass Filtering suppresses high-frequency components.
 4. High-Pass Filtering suppresses low-frequency components.
 5. Low-Pass Filtering is useful for smoothing and noise reduction.

6. High-Pass Filtering is useful for edge enhancement and feature extraction.
7. Frequency-domain filtering provides selective control over image frequencies.